



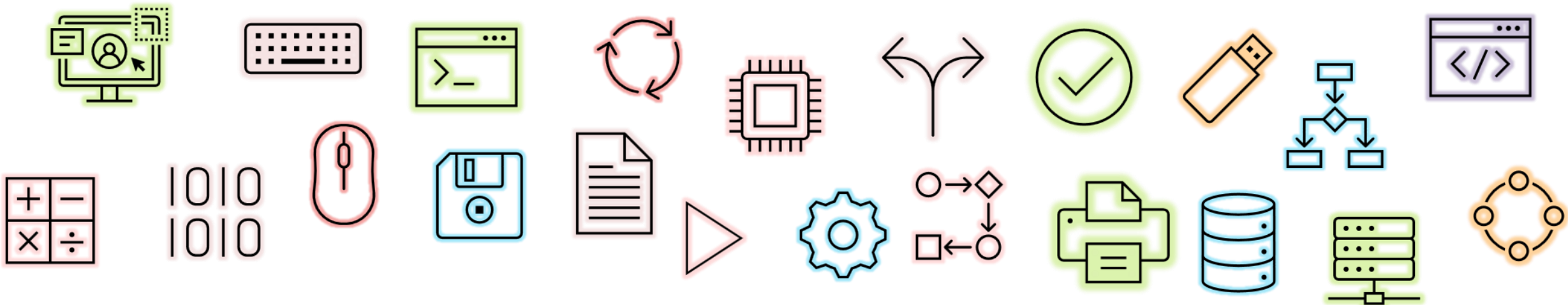
بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-202 Programming I

8- Multidimensional Arrays



Objectives

- To give examples of representing data using two-dimensional arrays (§8.1).
- To declare variables for two-dimensional arrays, create arrays, and access array elements in a two-dimensional array using row and column indexes (§8.2).
- To program common operations for two-dimensional arrays (displaying arrays, summing all elements, finding the minimum and maximum elements, and random shuffling) (§8.3).
- To pass two-dimensional arrays to methods (§8.4).
- To write a program for grading multiple-choice questions using two-dimensional arrays (§8.5).
- To solve the closest-pair problem using two-dimensional arrays (§8.6).
- To check a Sudoku solution using two-dimensional arrays (§8.7).
- To use multidimensional arrays (§8.8).

Motivations

- In Topic 7, we learned how to use basic one-dimensional arrays to store data
- You can also use two-dimensional arrays to store a matrix or a table of data

Distance Table (in miles)							
	Chicago	Boston	New York	Atlanta	Miami	Dallas	Houston
Chicago	0	983	787	714	1375	967	1087
Boston	983	0	214	1102	1763	1723	1842
New York	787	214	0	888	1549	1548	1627
Atlanta	714	1102	888	0	661	781	810
Miami	1375	1763	1549	661	0	1426	1187
Dallas	967	1723	1548	781	1426	0	239
Houston	1087	1842	1627	810	1187	239	0

Declaring & Creating Two-Dimensional arrays

- An element in a two-dimensional array is accessed through a row index and a column index

- Syntax:

```
elementType[][] arrayRefVar;
```

- Example to declare a 2D array of `int` values:

```
int[][] matrix;
```

- Of course, you can then create the actual array and assign the reference into the variable matrix

```
matrix = new int[5][5];
```

- This is a 5-by-5 (or 5x5) array of `int` values

Declaring & Creating Two-Dimensional arrays

- An element in a two-dimensional array is accessed through a row index and a column index
 - The new array is shown on the left below
 - And on the right, we save the value 7 into a cell
 - Which cell? Row 2 , Column 1.

[0][1][2][3][4]
[0] 0 0 0 0 0
[1] 0 0 0 0 0
[2] 0 0 0 0 0
[3] 0 0 0 0 0
[4] 0 0 0 0 0
matrix = new int[5][5];

[0][1][2][3][4]
[0] 0 0 0 0 0
[1] 0 0 0 0 0
[2] 0 7 0 0 0
[3] 0 0 0 0 0
[4] 0 0 0 0 0
matrix[2][1] = 7;

[0][1][2]
[0] 1 2 3
[1] 4 5 6
[2] 7 8 9
[3] 10 11 12
int[][] array = {
 {1, 2, 3},
 {4, 5, 6},
 {7, 8, 9},
 {10, 11, 12}
};

Declaring & Creating Two-Dimensional arrays

- Of course, you can declare and create in one step:

- Syntax:

```
int[][] matrix = new int[5][5];
```

- And like 1D arrays, you can use the array initializer
 - Here you declare, create, and initialize in one step:

```
int[][] array = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9},  
    {10, 11, 12}  
};
```

Equivalent

```
int[][] array = new int[4][3];  
array[0][0] = 1; array[0][1] = 2; array[0][2] = 3;  
array[1][0] = 4; array[1][1] = 5; array[1][2] = 6;  
array[2][0] = 7; array[2][1] = 8; array[2][2] = 9;  
array[3][0] = 10; array[3][1] = 11; array[3][2] = 12;
```

Lengths of Two-dimensional Arrays

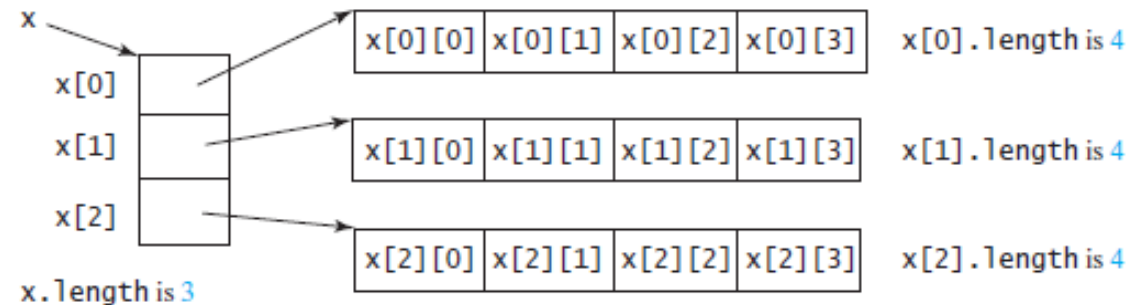
- A two-dimensional array is actually an array in which each element is a one-dimensional array!
- Length of an array, `x`, is the number of elements in the array
 - And each element is a row of the 2D array
 - You can get the number of rows by using `x.length`
- Then, `x[0]`, `x[1]`, ..., `x[x.length-1]` are arrays
 - You can get their lengths by using `x[0].length`, `x[1].length`, ..., and `x[x.length-1].length`

Lengths of Two-dimensional Arrays

- Example:

```
x = new int[3][4];
```

- Therefore, `x[0]`, `x[1]`, and `x[2]` are one-dimensional arrays, and each contains four elements



`x.length` is 3

`x[0].length`, `x[1].length`, and `x[2].length` is 4

Processing Two-Dimensional Arrays

- We never review several examples of array processing two-dimensional arrays:
 1. Initializing arrays with input values
 2. Printing arrays
 3. Summing all elements
 4. Summing all elements by column
 5. Which row has the largest sum
 6. Finding the smallest index of the largest element
 7. Random shuffling

	0	1	2	3	4
0					
1					
2					
3					

Processing Two-Dimensional Arrays

- Initializing arrays with input values
 - Of course, we use two **for** loops
 - Pay attention to the loop-continuation-condition
 - How we used **matrix.length**
 - and **matrix[row].length**

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
    for (int column = 0; column < matrix[row].length; column++) {
        matrix[row][column] = input.nextInt();
    }
}
```

	0	1	2	3	4
0					
1					
2					
3					

Processing Two-Dimensional Arrays

- Initializing arrays with **random** values
 - Same as last example
 - Same loop continuation conditions
 - Only difference is using random values

```
for (int row = 0; row < matrix.length; row++) {  
    for (int column = 0; column < matrix[row].length; column++) {  
        matrix[row][column] = (int)(Math.random() * 100);  
    }  
}
```

	0	1	2	3	4
0					
1					
2					
3					

Processing Two-Dimensional Arrays

- Printing arrays
 - We print the individual elements inside the inner `for` loop
 - And we use `print` (not `println`)
 - Then, after that loop finishes, we print a new line

```
for (int row = 0; row < matrix.length; row++) {  
    for (int column = 0; column < matrix[row].length; column++) {  
        System.out.print(matrix[row][column] + " ");  
    }  
    System.out.println();  
}
```

	0	1	2	3	4
0					
1					
2					
3					

Processing Two-Dimensional Arrays

- Summing all elements
 - We use a variable, `total`, to store the sum
 - Initially, `total` is `0`
 - We add each element of the 2D array to `total`
 - row by row, and column by column

```
int total = 0;
for (int row = 0; row < matrix.length; row++) {
    for (int column = 0; column < matrix[row].length; column++) {
        total += matrix[row][column];
    }
}
```

	0	1	2	3	4
0					
1					
2					
3					

Processing Two-Dimensional Arrays

- Summing all elements by column
 - For each column, we use the variable `total` to store the sum of elements in that specific column
- – Notice that we reset `total` to `0` after each iteration
- Thereby counting only the values in that column

```
for (int column = 0; column < matrix[0].length; column++) {  
    int total = 0;  
    for (int row = 0; row < matrix.length; row++)  
        total += matrix[row][column];  
    System.out.println("Sum for column " + column + " is " + total);  
}
```

	0	1	2	3	4
0					
1					
2					
3					

Processing Two-Dimensional Arrays

- Which row has the largest sum?
 - We use variables `maxRow` and `indexOfMaxRow`

```
int maxRow = 0;
int indexOfMaxRow = 0;
// Get sum of the first row in maxRow
for (int column = 0; column < matrix[0].length; column++) {
    maxRow += matrix[0][column];
}
for (int row = 1; row < matrix.length; row++) {
    int totalOfThisRow = 0;
    for (int column = 0; column < matrix[row].length; column++)
        totalOfThisRow += matrix[row][column];
    if (totalOfThisRow > maxRow) {
        maxRow = totalOfThisRow;
        indexOfMaxRow = row;
    }
}
System.out.println("Row " + indexOfMaxRow + " has the maximum sum of " + maxRow);
```

	0	1	2	3	4
0					
1					
2					
3					

Processing Two-Dimensional Arrays

- Random shuffling
 - We shuffled a 1D array in Topic 7
 - We generated a random index
 - And then we swapped the value at index "i" with the value at the randomly generated index
 - The idea for shuffling a 2D array is similar
 - Here, we generate two indices (**i1** and **j1**)
 - Then, we swap the value at **matrix[i][j]** with the value at **matrix[i1][j1]**

Processing Two-Dimensional Arrays

- Random shuffling

```
for (int i = 0; i < matrix.length; i++) {  
    for (int j = 0; j < matrix[i].length; j++) {  
        int i1 = (int)(Math.random() * matrix.length);  
        int j1 = (int)(Math.random() * matrix[i].length);  
        // Swap matrix[i][j] with matrix[i1][j1]  
        int temp = matrix[i][j];  
        matrix[i][j] = matrix[i1][j1];  
        matrix[i1][j1] = temp;  
    }  
}
```

Check Point

- What is the output of the following code?

```
int[][] array = {{1, 2}, {3, 4}, {5, 6}};
for (int i = array.length - 1; i >= 0; i--) {
    for (int j = array[i].length - 1; j >= 0; j--)
        System.out.print(array[i][j] + " ");
    System.out.println();
}
```

Check Point

- What is the output of the following code?

```
int[][] array = {{1, 2}, {3, 4}, {5, 6}};  
int sum = 0;  
for (int i = 0; i < array.length; i++)  
    sum += array[i][0];  
System.out.println(sum);
```

Passing 2D Arrays to Methods

- Always, when passing an array to a method, the **reference** of the array is passed
 - This is true for 1D arrays
 - This is true for 2D arrays
 - This is true for all arrays!
- You can also return a 2D array from a method
- Following example has two methods:
 1. `getArray()`: returns a 2D array
 2. `sum(int[][] m)`: returns sum of all elements

Passing 2D Arrays to Methods

LISTING 8.1 PassTwoDimensionalArray.java

```
1 import java.util.Scanner;
2
3 public class PassTwoDimensionalArray {
4     public static void main(String[] args) {
5         int[][] m = getArray(); // Get an array
6
7         // Display sum of elements
8         System.out.println("\nSum of all elements is " + sum(m));
9     }
10
11     public static int[][] getArray() {
12         // Create a Scanner
13         Scanner input = new Scanner(System.in);
14
15         // Enter array values
16         int[][] m = new int[3][4];
17         System.out.println("Enter " + m.length + " rows and "
18             + m[0].length + " columns: ");
19         for (int i = 0; i < m.length; i++)
20             for (int j = 0; j < m[i].length; j++)
21                 m[i][j] = input.nextInt();
22     }
```

```
23         return m;
24     }
25
26     public static int sum(int[][] m) {
27         int total = 0;
28         for (int row = 0; row < m.length; row++) {
29             for (int column = 0; column < m[row].length; column++) {
30                 total += m[row][column];
31             }
32         }
33
34         return total;
35     }
36 }
```

Passing 2D Arrays to Methods



Enter 3 rows and 4 columns:

1 2 3 4

5 6 7 8

9 10 11 12

Sum of all elements is 78

Program 1: Grade Exam

- Write a program to grade multiple-choice exams.
 - Assume there are eight students, and each exam has ten questions. The answers are stored in a 2D array. Each row represents the ten answers for one specific student. Write a program to grade the exam for each student.
- Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 1: Grade Exam

- Step 1: Problem-solving Phase
 - Details
 - Here is a representation of the student grades:

	0	1	2	3	4	5	6	7	8	9
Student 0	A	B	A	C	C	D	E	E	A	D
Student 1	D	B	A	B	C	A	E	E	A	D
Student 2	E	D	D	A	C	B	E	E	A	D
Student 3	C	B	A	E	D	C	E	E	A	D
Student 4	A	B	D	C	C	D	E	E	A	D
Student 5	B	B	E	C	C	D	E	E	A	D
Student 6	B	B	A	C	C	D	E	E	A	D
Student 7	E	B	E	C	C	D	E	E	A	D

Program 1: Grade Exam

- Step 1: Problem-solving Phase
 - Details
 - The answer key is stored in a one-dimensional array:

Key to the Questions:

0 1 2 3 4 5 6 7 8 9

Key D B D C C D A E A D

- So our program must use two nested **for** loops
- For each student (row), we must compare the values in that row to the values in the answer key

Program 1: Grade Exam

LISTING 8.2 GradeExam.java

```
1 public class GradeExam {
2     /** Main method */
3     public static void main(String[] args) {
4         // Students' answers to the questions
5         char[][] answers = {                                2-D array
6             {'A', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
7             {'D', 'B', 'A', 'B', 'C', 'A', 'E', 'E', 'A', 'D'},
8             {'E', 'D', 'D', 'A', 'C', 'B', 'E', 'E', 'A', 'D'},
9             {'C', 'B', 'A', 'E', 'D', 'C', 'E', 'E', 'A', 'D'},
10            {'A', 'B', 'D', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
11            {'B', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
12            {'B', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
13            {'E', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'}};
14
15        // Key to the questions
16        char[] keys = {'D', 'B', 'D', 'C', 'C', 'D', 'A', 'E', 'A', 'D'};  1-D array
17
18        // Grade all answers
19        for (int i = 0; i < answers.length; i++) {
20            // Grade one student
21            int correctCount = 0;
22            for (int j = 0; j < answers[i].length; j++) {
23                if (answers[i][j] == keys[j])                compare with key
24                    correctCount++;
25            }
26
27            System.out.println("Student " + i + "'s correct count is " +
28                               correctCount);
29        }
30    }
31 }
```

Program 1: Grade Exam

- Run the Program:



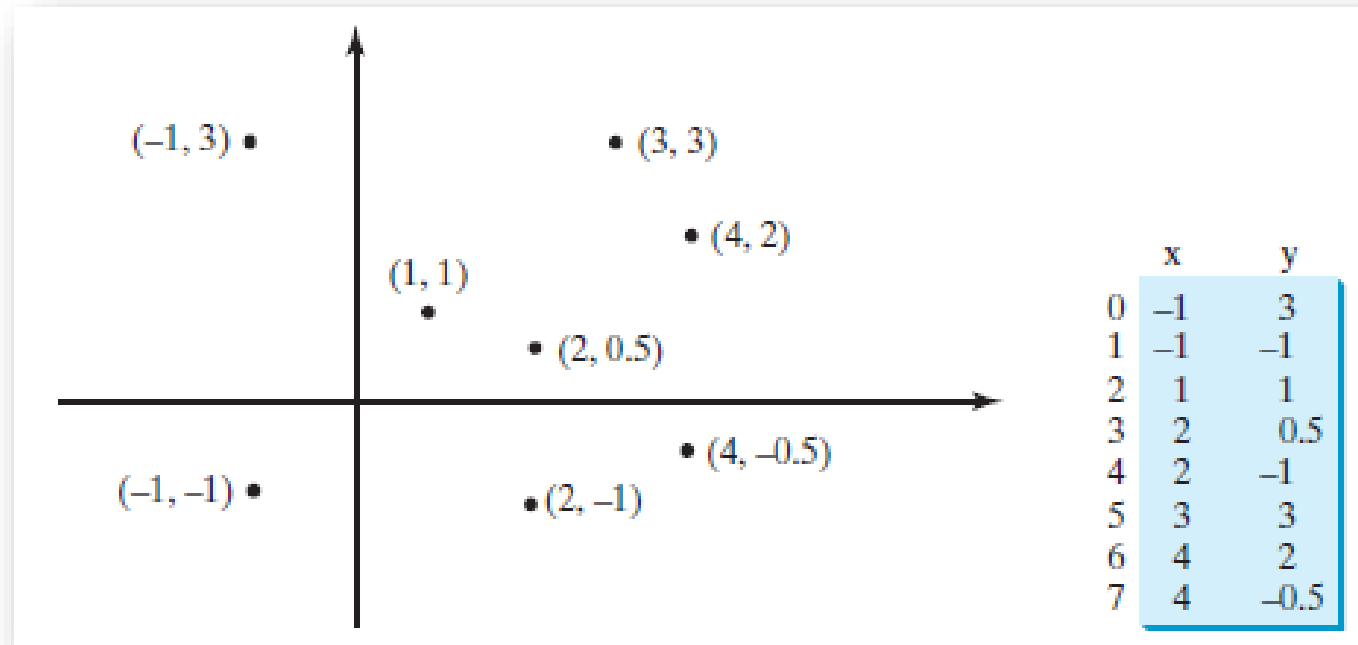
```
Student 0's correct count is 7  
Student 1's correct count is 6  
Student 2's correct count is 5  
Student 3's correct count is 4  
Student 4's correct count is 8  
Student 5's correct count is 7  
Student 6's correct count is 7  
Student 7's correct count is 7
```

Program 2: Find Nearest Points

- Given a set of points, find the two points that are closest to each other.
- Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 2: Find Nearest Points

- Step 1: Problem-solving Phase
 - Details
 - In the example below, points (1, 1) and (2, 0.5) are closest



Program 2: Find Nearest Points

- Step 1: Problem-solving Phase
 - Algorithm:
 - We read the coordinates from the user
 - Save them into a 2D array
 - Find the distance between each and every pair of points
 - Choose the distance that is smallest
 - Specifically, we save the smallest distance, and we save the points that provided this smallest distance
 - How do we calculate distance?
 - We use the famous Euclidean distance formula
 - $distance = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

Program 2: Find Nearest Points

LISTING 8.3 FindNearestPoints.java

```
1 import java.util.Scanner;
2
3 public class FindNearestPoints {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6         System.out.print("Enter the number of points: ");
7         int numberOfPoints = input.nextInt();
8
9         // Create an array to store points
10        double[][] points = new double[numberOfPoints][2];
11        System.out.print("Enter " + numberOfPoints + " points: ");
12        for (int i = 0; i < points.length; i++) {
13            points[i][0] = input.nextDouble();
14            points[i][1] = input.nextDouble();
15        }
16
17        // p1 and p2 are the indices in the points' array
18        int p1 = 0, p2 = 1; // Initial two points
19        double shortestDistance = distance(points[p1][0], points[p1][1],
20            points[p2][0], points[p2][1]); // Initialize shortestDistance
21    }
```

```
22        // Compute distance for every two points
23        for (int i = 0; i < points.length; i++) {
24            for (int j = i + 1; j < points.length; j++) {
25                double distance = distance(points[i][0], points[i][1],
26                    points[j][0], points[j][1]); // Find distance
27
28                if (shortestDistance > distance) {
29                    p1 = i; // Update p1
30                    p2 = j; // Update p2
31                    shortestDistance = distance; // Update shortestDistance
32                }
33            }
34        }
35
36        // Display result
37        System.out.println("The closest two points are " +
38            "(" + points[p1][0] + ", " + points[p1][1] + ") and (" +
39            points[p2][0] + ", " + points[p2][1] + ")");
40    }
41
42    /** Compute the distance between two points (x1, y1) and (x2, y2) */
43    public static double distance(
44        double x1, double y1, double x2, double y2) {
45        return Math.sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
46    }
47 }
```

Program 2: Find Nearest Points

- Run the Program:

```
Enter the number of points: 8 ↵ Enter
Enter 8 points: -1 3 -1 -1 1 1 2 0.5 2 -1 3 3 4 2 4 -0.5 ↵ Enter
The closest two points are (1, 1) and (2, 0.5)
```





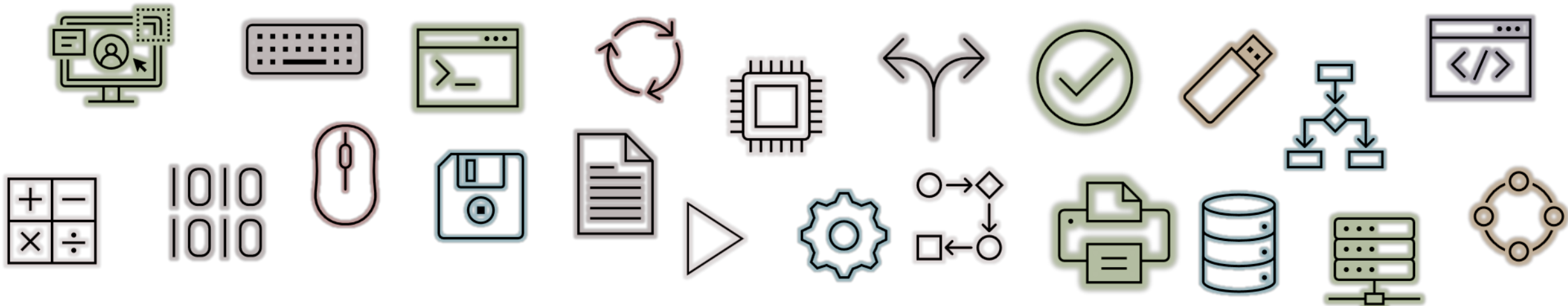
بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-202 Programming I

6- Methods



Program 3: Sudoku

- Given a possible solution to Sudoku, your program must check to see if the solution is correct.
- Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 3: Sudoku

- Step 1: Problem-solving Phase
 - Details of Sudoku
 - Sudoku is a 9×9 grid, which is divided into 3×3 boxes
 - Also called regions or blocks
 - So there are 81 total cells, and 9 cells in each 3×3 box
 - Some of the cells are already filled-in with numbers 1 – 9
 - The goal of the problem is to fill the empty cells
 - You can only use the numbers 1 – 9
 - Inside each row, there cannot be duplicate numbers
 - Inside each column, there cannot be duplicate numbers
 - Inside each 3×3 region, there cannot be duplicate numbers

Program 3: Sudoku

- Step 1: Problem-solving Phase
 - Details of Sudoku

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6							
			4	1	9			5
				8			7	9
(a) Puzzle								
Solution →								
5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9
(b) Solution								

Program 3: Sudoku

- Step 1: Problem-solving Phase
 - Details:
 - Your job is to check if a given solution is valid
 - Note that we use 0 to indicate a free/empty cell
 - The Sudoku grid can be naturally represented with a 2D array

5	3	0	0	7	0	0	0	0
6	0	0	1	9	5	0	0	0
0	9	8	0	0	0	0	6	0
8	0	0	0	6	0	0	0	3
4	0	0	8	0	3	0	0	1
7	0	0	0	2	0	0	0	6
0	6	0	0	0	0	0	0	0
0	0	0	4	1	9	0	0	5
0	0	0	0	8	0	0	7	9

(a)

```
int[][] grid =  
{ {5, 3, 0, 0, 7, 0, 0, 0, 0},  
  {6, 0, 0, 1, 9, 5, 0, 0, 0},  
  {0, 9, 8, 0, 0, 0, 0, 6, 0},  
  {8, 0, 0, 0, 6, 0, 0, 0, 3},  
  {4, 0, 0, 8, 0, 3, 0, 0, 1},  
  {7, 0, 0, 0, 2, 0, 0, 0, 6},  
  {0, 6, 0, 0, 0, 0, 0, 0, 0},  
  {0, 0, 0, 4, 1, 9, 0, 0, 5},  
  {0, 0, 0, 0, 8, 0, 0, 7, 9}  
};
```

(b)

Program 3: Sudoku

- Step 1: Problem-solving Phase
 - Details:
 - Assume we have a possible solution
 - Meaning, each zero would be replaced with an appropriate number between 1 and 9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

(b) Solution

A solution grid is

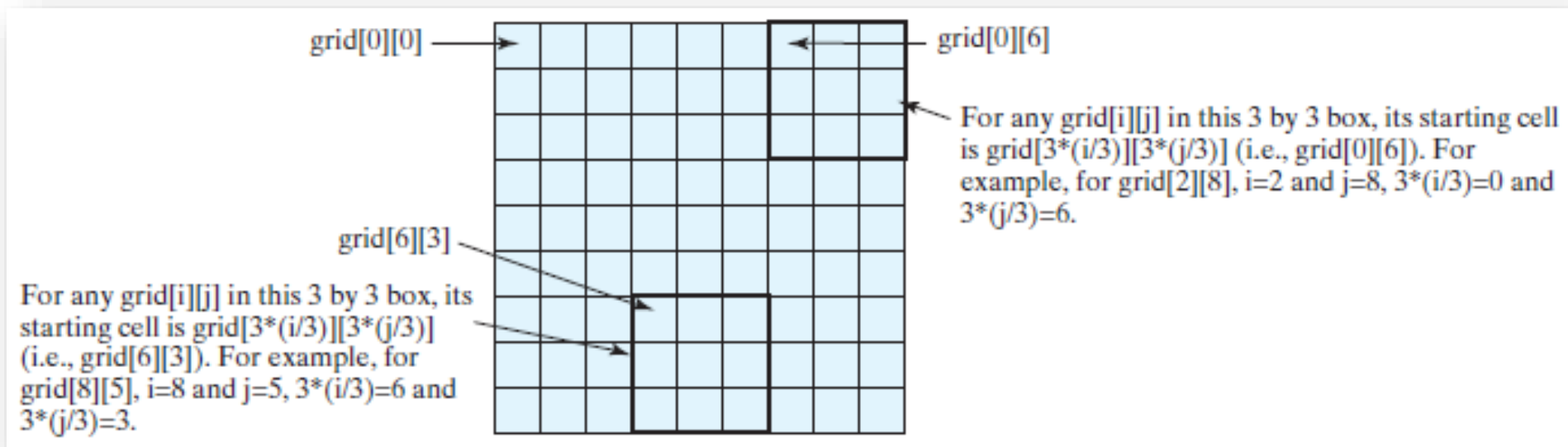
```
{{5, 3, 4, 6, 7, 8, 9, 1, 2},  
 {6, 7, 2, 1, 9, 5, 3, 4, 8},  
 {1, 9, 8, 3, 4, 2, 5, 6, 7},  
 {8, 5, 9, 7, 6, 1, 4, 2, 3},  
 {4, 2, 6, 8, 5, 3, 7, 9, 1},  
 {7, 1, 3, 9, 2, 4, 8, 5, 6},  
 {9, 6, 1, 5, 3, 7, 2, 8, 4},  
 {2, 8, 7, 4, 1, 9, 6, 3, 5},  
 {3, 4, 5, 2, 8, 6, 1, 7, 9}  
};
```

Program 3: Sudoku

- Step 1: Problem-solving Phase
 - Algorithm:
 - How do we verify a Sudoku solution is valid?
 - There are two approaches:
 - Check if every row has numbers from 1 to 9, every column has numbers from 1 to 9, and every small box has numbers from 1 to 9
 - Check each cell. Each cell must be a number from 1 to 9, and the cell must be unique on every row, every column, and every small box
 - We use the second approach for our Sudoku solver

Program 3: Sudoku

- Step 1: Problem-solving Phase
 - Algorithm:
 - Checking if a cell is unique in the row and column is easy
 - But how to check if a cell is unique in the 3x3 region?



Program 3: Sudoku

- Step 1: Problem-solving Phase
 - Algorithm:
 - Using that observation, if `grid[r][c]` is the starting cell of a 3×3 box, the cells in that box can be checked by using a nested loop as follows:

```
// Get all cells in a 3-by-3 box starting at grid[r][c]
for (int row = r; row < r + 3; row++)
    for (int col = c; col < c + 3; col++)
        // grid[row][col] is in the box
```

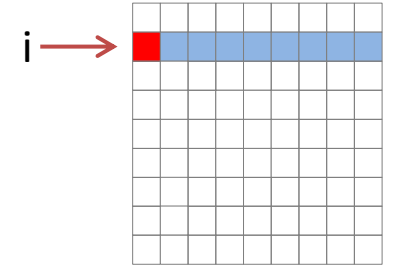
Program 3: Sudoku

```
// Check whether grid[i][j] is unique in i's row
for (int column = 0; column < 9; column++)
    if (column != j && grid[i][column] == grid[i][j])
        return false;
```

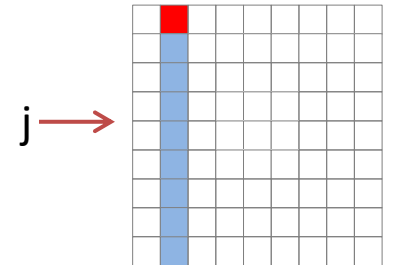
grid[i][j]

5 2 8 5 4 9 7 1 3

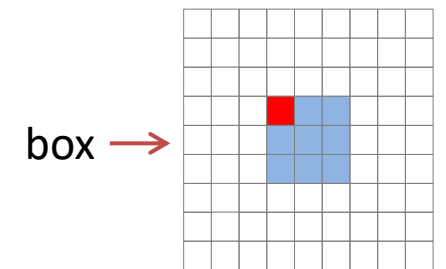
5 4 1 9 6 7 8 2 3



```
// Check whether grid[i][j] is unique in j's column
for (int row = 0; row < 9; row++)
    if (row != i && grid[row][j] == grid[i][j])
        return false;
```



```
// Check whether grid[i][j] is unique in the 3-by-3 box
for (int row = (i / 3) * 3; row < (i / 3) * 3 + 3; row++)
    for (int col = (j / 3) * 3; col < (j / 3) * 3 + 3; col++)
        if (row != i && col != j && grid[row][col] == grid[i][j])
            return false;
```

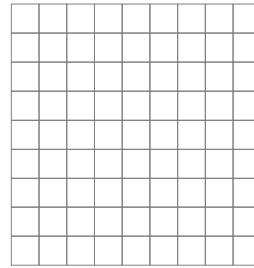


Program 3: Sudoku

- Step 2: Implementation

LISTING 8.4 CheckSudokuSolution.java

```
1 import java.util.Scanner;
2
3 public class CheckSudokuSolution {
4     public static void main(String[] args) {
5         // Read a Sudoku solution
6         int[][] grid = readASolution();
7
8         System.out.println(isValid(grid) ? "Valid solution" :
9             "Invalid solution");
10    }
11
12    /** Read a Sudoku solution from the console */
13    public static int[][] readASolution() {
14        // Create a Scanner
15        Scanner input = new Scanner(System.in);
16
17        System.out.println("Enter a Sudoku puzzle solution:");
18        int[][] grid = new int[9][9];
19        for (int i = 0; i < 9; i++)
20            for (int j = 0; j < 9; j++)
21                grid[i][j] = input.nextInt();
22
23        return grid;
24    }
25
26    /** Check whether a solution is valid */
27    public static boolean isValid(int[][] grid) {
```



```
28     for (int i = 0; i < 9; i++)
29         for (int j = 0; j < 9; j++)
30             if (grid[i][j] < 1 || grid[i][j] > 9
31                 || !isValid(i, j, grid))
32                 return false;
33     return true; // The solution is valid
34 }
35
36 /** Check whether grid[i][j] is valid in the grid */
37 public static boolean isValid(int i, int j, int[][] grid) {
38     // Check whether grid[i][j] is unique in i's row
39     for (int column = 0; column < 9; column++)
40         if (column != j && grid[i][column] == grid[i][j])
41             return false;
42
43     // Check whether grid[i][j] is unique in j's column
44     for (int row = 0; row < 9; row++)
45         if (row != i && grid[row][j] == grid[i][j])
46             return false;
47
48     // Check whether grid[i][j] is unique in the 3-by-3 box
49     for (int row = (i / 3) * 3; row < (i / 3) * 3 + 3; row++)
50         for (int col = (j / 3) * 3; col < (j / 3) * 3 + 3; col++)
51             if (row != i && col != j && grid[row][col] == grid[i][j])
52                 return false;
53
54     return true; // The current value at grid[i][j] is valid
55 }
56 }
```

Program 3: Sudoku



Enter a Sudoku puzzle solution:

9 6 3 1 7 4 2 5 8	↵ Enter
1 7 8 3 2 5 6 4 9	↵ Enter
2 5 4 6 8 9 7 3 1	↵ Enter
8 2 1 4 3 7 5 9 6	↵ Enter
4 9 6 8 5 2 3 1 7	↵ Enter
7 3 5 9 6 1 8 2 4	↵ Enter
5 8 9 7 1 3 4 6 2	↵ Enter
3 1 7 2 4 6 9 8 5	↵ Enter
6 4 2 5 9 8 1 7 3	↵ Enter

Valid solution

Multidimensional Arrays

- With Java, you can also create n-dimensional arrays for any integer n
- Example:
 - Suppose you have a class of **six** students
 - Each student has **five** exam grades
 - So this sounds like a 2D array

Multidimensional Arrays

- With Java, you can also create n-dimensional arrays for any integer n
- Example:
 - Suppose you have a class of **six** students
 - Each student has **five** exam grades
 - So this sounds like a 2D array
 - Rows are six students and the columns are five exam grades

	grade				
	0	1	2	3	4
student 0					
1					
2					
3					
4					
5					

Multidimensional Arrays

- With Java, you can also create n-dimensional arrays for any integer n
- Example:
 - Suppose you have a class of **six** students
 - Each student has **five** exam grades
 - So this sounds like a 2D array
 - Rows are six students and the columns are five exam grades
 - However, each exam has **two** graded parts
 - Multiple choice (MC) and essay section

	grade				
	0	1	2	3	4
student 0					
1					
2					
3					
4					
5					

Multidimensional Arrays

- With Java, you can also create n-dimensional arrays for any integer n
- Example:
 - Suppose you have a class of **six** students
 - Each student has **five** exam grades
 - So this sounds like a 2D array
 - Rows are six students and the columns are five exam grades
 - However, each exam has **two** graded parts
 - Multiple choice (MC) and essay section
 - For this example, we need a **three-dimensional array**!

	0	1	2	3	4	
0						
1						
2						
3						
4						
5						

student

	0	1	2	3	4	
0						
1						
2						
3						
4						
5						

grade

part

Multidimensional Arrays

- Example:

(student, grade, part)

```
double[][][] scores = new double[6][5][2];
```

- You can also use short-hand notation to create and
- Initialize in one step:

```
double[][][] scores = {  
    {{7.5, 20.5}, {9.0, 22.5}, {15, 33.5}, {13, 21.5}, {15, 2.5}},  
    {{4.5, 21.5}, {9.0, 22.5}, {15, 34.5}, {12, 20.5}, {14, 9.5}},  
    {{6.5, 30.5}, {9.4, 10.5}, {11, 33.5}, {11, 23.5}, {10, 2.5}},  
    {{6.5, 23.5}, {9.4, 32.5}, {13, 34.5}, {11, 20.5}, {16, 7.5}},  
    {{8.5, 26.5}, {9.4, 52.5}, {13, 36.5}, {13, 24.5}, {16, 2.5}},  
    {{9.5, 20.5}, {9.4, 42.5}, {13, 31.5}, {12, 20.5}, {16, 6.5}}};
```

- `scores[0][1][0]` refers to score for student exam
- `scores[0][1][1]` refers to score for student

Multidimensional Arrays

- Example:

(student, grade, part)

```
double[][][] scores = new double[6][5][2];
```

- You can also use short-hand notation to create and
- Initialize in one step:

```
double[][][] scores = {  
    {{7.5, 20.5}, {9.0, 22.5}, {15, 33.5}, {13, 21.5}, {15, 2.5}},  
    {{4.5, 21.5}, {9.0, 22.5}, {15, 34.5}, {12, 20.5}, {14, 9.5}},  
    {{6.5, 30.5}, {9.4, 10.5}, {11, 33.5}, {11, 23.5}, {10, 2.5}},  
    {{6.5, 23.5}, {9.4, 32.5}, {13, 34.5}, {11, 20.5}, {16, 7.5}},  
    {{8.5, 26.5}, {9.4, 52.5}, {13, 36.5}, {13, 24.5}, {16, 2.5}},  
    {{9.5, 20.5}, {9.4, 42.5}, {13, 31.5}, {12, 20.5}, {16, 6.5}}};
```

- scores[0][1][0] refers to MC score for first student 2nd exam
- scores[0][1][1] refers to essay score for that same student



بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-202 Programming I

END

